SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

for

TasteTrail – Smart Restaurant Discovery & Recommendation Platform

A Web-Based Platform for Discovering, Comparing and Navigating to Restaurants

Document Version 1.0

Document Status: Final – For Faculty Evaluation

Prepared For: GLA University

Course / Program: B.TECH CSE 3rd Semester

Prepared By: Vania Marwah(22515000150)

Project Guide: Mr. Gautam Mukherjee

Date: 18/08/2026

Prepared in accordance with IEEE 830 / ISO/IEC/IEEE 29148 SRS guidelines

Table of Contents

Section	Page
1. Introduction	3
1.1 Purpose	3
1.2 Document Conventions	3
1.3 Intended Audience and Reading Suggestions	3
1.4 Project Scope	3
1.5 References	3
2. Overall Description	4
2.1 Product Perspective	4
2.2 Product Features (Summary)	4
2.3 User Classes and Characteristics	4
2.4 Operating Environment	4
2.5 Design and Implementation Constraints	5
2.6 Assumptions and Dependencies	5
3. System Features (Functional Requirements)	5
3.1 Module: User & Location	5
3.2 Module: Restaurant Discovery & Search	6
3.3 Module: Restaurant Details & Comparison	6
3.4 Module: Map & Navigation	7
3.5 Module: Favorites, Reviews & Recommendations	7
4. External Interface Requirements	8
4.1 User Interfaces	8
4.2 Hardware Interfaces	8
4.3 Software Interfaces	8
4.4 Communication Interfaces	8
5. Use Case Summary	9
6. Non-Functional Requirements	10
6.1 Performance Requirements	10
6.2 Security Requirements	10
6.3 Usability	10
6.4 Reliability & Availability	10
6.5 Maintainability & Portability	11
6.6 Scalability	11
7. Software Testing Considerations	11
8. Proposed System Architecture	12
8.1 Suggested Technology Stack	12
9. Data Model Overview	13
9.1 Core Entities	13
9.2 Key Relationships	13

Section	Page
10. Future Enhancements	14
11. Glossary	14

1. Introduction

1.1 Purpose

ThisSoftware Requirements Specification (SRS) document describes the functional and non-functional requirements for TasteTrail, a web-based restaurant discovery and recommendation platform designed to help users find suitable nearby restaurants based on location, cuisine, budget, dining preferences, and other relevant factors. The document serves as the primary reference for project development, UI/UX design, software testing, and faculty evaluation.

1.2 Document Conventions

Functionalrequirementsareuniquely identified using the format FR-XX and grouped by module. Each requirement uses the keyword “shall” to indicate a mandatory capability. Priority levels are High (must be implemented), Medium (important but may be simplified), and Low (optional or future enhancement).

1.3 Intended Audience and Reading Suggestions

- Students / DevelopmentTeam – to understandthe systemmodules and required functionality.
- Project Guide / Faculty Evaluator – to assess project completeness and feasibility.
- UI/UX Designer – to design screens and user workflows.
- Software Tester – to derive test cases from the functional requirements.

1.4 Project Scope

TasteTrailwillallow users to discover nearby restaurants, search and filter them according to cuisine, budget, distance, dietary preferences and dining purpose, view restaurant details, compare restaurants, save favourites, and view restaurants on an interactive map. The platform will provide route information and a recommendation-oriented interface to help users make an informed dining choice. The academic version focuses primarily on the frontend experience and may use mock or publicly available restaurant/location data.

1.5 References

- IEEEStd 830-1998 – IEEE Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018 – Systems and Software Engineering – Requirements Engineering.
- React.js, Vite, Tailwind CSS and Leaflet documentation.
- Projectsynopsis / descriptionsubmitted by the student.

2. Overall Description

2.1 Product Perspective

TasteTrail is a new, self-contained web application. It is designed as a user-facing restaurant discovery platform and can initially operate using mock restaurant data or external location/place APIs. The architecture is kept modular so that live restaurant information, authentication, reviews and recommendation services can be integrated in future versions.

2.2 Product Features (Summary)

- Location-basednearby restaurant discovery.
- Search and advanced filtering by cuisine, budget, distance, rating and preferences.
- Restaurant cards and detailed restaurant profiles.
- Interactive map with restaurant markers.
- Route and distance information for reaching a selected restaurant.
- Restaurant comparison feature.
- Mood/dining-purpose based recommendations.
- Favourite/saved restaurant list.
- Ratings, reviews and trust-oriented information display.

• Trending and recommended restaurant sections.

2.3 User Classes and Characteristics

User Class	Description	Technical Characteristics
Visitor / User	Person searching for restaurants in a new or familiar location.	Basic smartphone/web literacy; primary user.
Registered User	User who saves favourites, preferences and reviews.	Basic to moderate technical familiarity.
Administrator	Trusted user who manages restaurant/content data in an expanded version.	Moderate technical familiarity; administrative access.

2.4 Operating Environment

• Client side: Modern web browsers such as Chrome, Firefox, Edge and Safari on desktop, laptop, tablet or smartphone.
• Development environment: Node.js with Vite and React.js.
• Data source: Mock JSON data for the academic frontend prototype and optional public location/place APIs.
• Map service: Leaflet/OpenStreetMap or another approved map provider.

2.5 Design and Implementation Constraints

• The project must be completed within a college semester.
• The primary implementation shall use free or open-source technologies where possible.
• The interface shall be responsive across desktop and mobile devices.
• Live API usage may depend on API limits, keys, network connectivity and provider policies.
• The current academic scope is primarily frontend-focused; persistent backend services are optional/future scope.

2.6 Assumptions and Dependencies

• Users provide or permit access to a location for nearby search, where supported.
• Restaurant data is available through mock data or an approved external API.
• Map and route information depends on the selected map/location provider.
• Internet connectivity is available for live map and API-based features.

3. System Features (Functional Requirements)

3.1 Module: User & Location

Description: Provides the basic user context and location required for restaurant discovery.

ID	Requirement	Priority
FR-01	The system shall allow a user to enter a location manually or use the current location where browser permissions are granted.	High
FR-02	The system shall display the selected/current location clearly on the discovery interface.	High
FR-03	The system shall allow a user to search restaurants around a selected location.	High
FR-04	The system shall allow a registered user to save and update basic dining preferences.	Medium
FR-05	The system shall provide clear feedback when location access is denied or unavailable.	High

3.2 Module: Restaurant Discovery & Search

ID	Requirement	Priority
FR-06	The system shall display a list/grid of nearby restaurants with essential information such as name, cuisine, rating, distance and price range.	High

ID	Requirement	Priority
FR-07	The system shall allow users to search restaurants by name, cuisine or keyword.	High
FR-08	The system shall allow users to filter restaurants by cuisine, budget, rating, distance and availability/open status.	High
FR-09	The system shall support additional preference filters such as vegetarian, vegan, family-friendly, outdoor seating and similar attributes when data is available.	Medium
FR-10	The system shall allow users to sort results by relevance, rating, distance and price.	Medium
FR-11	The system shall display an empty-state message when no restaurant matches the selected criteria.	High

3.3 Module: Restaurant Details & Comparison

ID	Requirement	Priority
FR-12	The system shall display a detailed restaurant profile including name, photos, cuisine, price range, rating, address, opening hours and facilities.	High
FR-13	The system shall display menu highlights or signature dishes when available in the dataset.	Medium
FR-14	The system shall allow users to view available ratings and review summaries.	Medium
FR-15	The system shall allow users to select two or more restaurants for side-by-side comparison.	Medium
FR-16	The comparison view shall show relevant differences such as rating, price, distance, cuisine and facilities.	Medium

3.4 Module: Map & Navigation

ID	Requirement	Priority
FR-17	The system shall display restaurant locations using an interactive map.	High
FR-18	The system shall allow a user to select a restaurant marker and open its summary/details.	High
FR-19	The system shall display the distance between the user's selected location and a restaurant where location data is available.	High
FR-20	The system shall provide a route/navigation action for reaching the selected restaurant through the supported map service.	High
FR-21	The system shall provide a fallback message when map or route data cannot be loaded.	Medium

3.5 Module: Favorites, Reviews & Recommendations

ID	Requirement	Priority
FR-22	The system shall allow a registered user to add or remove restaurants from a favourites list.	Medium
FR-23	The system shall display a user's saved/favourite restaurants.	Medium
FR-24	The system shall provide recommendation sections based on selected preferences, dining purpose or mood using available restaurant data.	Medium
FR-25	The system shall allow a user to select a dining purpose such as family dinner, date night, quick meal, work/study or late-night dining.	Medium
FR-26	The system shall display a trust-oriented summary using available rating/review information without claiming that the score is an official rating.	Medium
FR-27	The system shall provide clear prompts for unavailable, incomplete or outdated restaurant information.	Low

4. External Interface Requirements

4.1 User Interfaces

• Landing / Home Page – search/location entry, featured restaurants, quick filters and recommendations.
• Explore Page – restaurant cards, search, filters, sorting and map view.
• Restaurant Details Page – photos, information, reviews, facilities, menu highlights and navigation action.
• Compare Page – side-by-side restaurant comparison.
• Favorites / Profile – saved restaurants and preference settings.
• The application shall maintain a consistent, modern, responsive layout suitable for desktop and mobile screens.

4.2 Hardware Interfaces

Nospecialized hardwareisrequired. The system runs on standard desktops, laptops, tablets and smartphones. A GPS/location-capable device may be used for current-location functionality.

4.3 Software Interfaces

Component	Technology
Frontend	React.js, React Router, Vite, Tailwind CSS
Maps	Leaflet with OpenStreetMap or an approved map provider
Charts / Visualization	Recharts or another suitable frontend charting library
Data	Mock JSON/local data; optional REST API for live restaurant/location data

Component	Technology
Version Control	Git & GitHub

4.4 Communication Interfaces

• Ifexternal APIs are used, client-siderequests shall use HTTPS.
• REST/JSON APIs may be used for restaurant, geolocation, mapping or routing data.
• Browser geolocation APIs shall be accessed only after appropriate user permission.

5. Use Case Summary

Actor	Use Case	Brief Description
User	Set Location	Enter a location or use current location to begin restaurant discovery.
User	Search / Filter Restaurants	Find restaurants using cuisine, budget, distance, rating and other preferences.
User	View Restaurant	Open a detailed restaurant profile and inspect key information.
User	Compare Restaurants	Compare selected restaurants using common criteria.
User	View Map / Route	Locate a restaurant and open route/navigation information.
User	Save Favourite	Save restaurants for later reference.
User	Get Recommendations	Receive suggestions based on mood, purpose or preferences.

5.1 Primary Flow – Restaurant Discovery

1) User opens TasteTrail →2) Selects current/manual location → 3) Searches or chooses preferences → 4) System displays matching restaurants → 5) User filters/sorts results → 6) User opens a restaurant profile → 7) User views map/location and route → 8) User saves, compares or chooses the restaurant.

6. Non-Functional Requirements

6.1 Performance Requirements

ID	Requirement	Priority
NFR-01	The system shall load core application pages within approximately 3 seconds under normal network conditions.	High
NFR-02	Search and filtering interactions shall provide visible results without unnecessary page reloads.	High
NFR-03	The interface shall remain responsive while rendering restaurant cards and map elements.	Medium

6.2 Security Requirements

ID	Requirement	Priority
NFR-04	The application shall not expose API keys or sensitive credentials in publicly accessible source code where the selected API provider requires protected credentials.	High
NFR-05	User-provided search/filter input shall be validated and safely handled.	High
NFR-06	Location access shall require explicit browser permission and shall not be requested unnecessarily.	High

6.3 Usability

• The interface shall be simple enough for a first-time user to find a suitable restaurant quickly.
• The system shall be fully responsive across desktop, tablet and mobile screens.
• Search, filters and navigation actions shall be clearly visible and understandable.
• Error, loading and empty-state messages shall be specific and easy to understand.

6.4 Reliability & Availability

• The frontend shall handle unavailable API/map data gracefully without crashing.
• Mock/local data shall allow the academic demonstration to continue even when external services are unavailable.

6.5 Maintainability & Portability

• The codebase shall use reusable React components and modular pages/services.
• The system shall run on Windows, Linux or macOS using a modern Node.js environment and shall be accessible through modern web browsers.

6.6 Scalability

The application shall be structured so that additional cities, restaurant categories, user accounts, live APIs, recommendation services and backend storage can be added without redesigning the complete frontend.

7. Software Testing Considerations

Each functional requirement in Section 3 should map to at least one test case. Representative test cases are provided below.

Test ID	Req.	Test Scenario	Expected Result
TC-01	FR-01	Allow location access / enter a valid location	Selected location is accepted and nearby results can be loaded.
TC-02	FR-08	Apply cuisine and budget filters	Only matching restaurants are displayed.
TC-03	FR-12	Open a restaurant card	Complete restaurant details are displayed.
TC-04	FR-17	Open map view	Restaurant markers are displayed at valid locations.
TC-05	FR-20	Select route/navigation action	Supported map service opens route information.
TC-06	FR-15	Compare two restaurants	Selected restaurants appear side-by-side with comparable attributes.

Recommended testing types: component/unit testing for reusable UI components, functional testing for search/filter/navigation flows, integration testing for external APIs where used, and manual responsive UI/UX testing across at least two browsers and one mobile viewport.

8. Proposed System Architecture

TasteTrail is proposed as a frontend-centered web architecture. The presentation layer contains the React user interface and reusable components. A service/data layer handles mock JSON data and optional external REST APIs for restaurant, geolocation and map information. A backend/database can be introduced in a future phase for persistent accounts, reviews and favourites.

Presentation Layer

React.js SPA – Home / Explore / Restaurant Details / Map / Compare / Favorites UI

↓ HTTPS / REST / Browser APIs

Service / Data Layer

Mock JSON data + optional restaurant/place APIs + browser Geolocation + Map/Routing service

↓ Future integration
Optional future Node.js/Express API + database for users, reviews, favourites and restaurant data.

Application / Data Layer

8.1 Suggested Technology Stack

Layer	Technology
Frontend	React.js, Vite, React Router, Tailwind CSS
Maps / Routing	Leaflet / OpenStreetMap or approved mapping provider
Charts	Recharts or equivalent
Data	JSON / local mock data; optional REST APIs
Version Control	Git & GitHub
Testing	Browser DevTools, manual UI testing, optional Jest/Vitest
Deployment	Vercel / Netlify or equivalent static hosting

9. Data Model Overview (for Database Planning)

The academic frontend may use mock JSON objects. The following entities are proposed as a starting point for future persistent storage.

9.1 Core Entities

Entity	Key Attributes
User	id, name, email, preferences, location, createdAt
Restaurant	id, name, cuisine, address, latitude, longitude, priceRange, rating, hours, facilities
MenuItem	id, restaurantId, name, category, price, isSignature
Review	id, restaurantId, userId, rating, comment, createdAt
Favorite	id, userId, restaurantId, createdAt
Recommendation	id, restaurantId, criteria, score / reason

9.2 Key Relationships

- A Restaurant has many MenuItems (one-to-many).
- A Restaurant can have many Reviews (one-to-many).
- A User can save many Restaurants through Favorites (many-to-many).
- A User can create many Reviews (one-to-many).
- A Recommendation references a Restaurant and is generated from selected criteria/preferences.

10. Future Enhancements (Out of Current Scope)

- Real-time restaurant availability and estimated waiting time.
- User authentication and cloud-synchronised favourites.
- AI/ML-based personalized restaurant recommendation.
- Sentiment analysis of restaurant reviews.
- Verified/reliable review scoring and fraud detection.
- Live table reservation and online ordering integration.
- Multi-city and multilingual support.
- Restaurant-owner portal for updating menus and offers.
- Real-time traffic-aware route recommendations.
- Voice-based restaurant search.

11. Glossary

Term	Definition
SRS	Software Requirements Specification
API	Application Programming Interface
REST	Representational State Transfer – an architectural style commonly used for web APIs
UI/UX	User Interface / User Experience
GPS	Global Positioning System used to determine geographic location
CRUD	Create, Read, Update, Delete – basic data operations
SPA	Single Page Application – a web application that updates content without full page reloads
JSON	JavaScript Object Notation – a common data interchange format

End of Document – TasteTrail SRS v1.0